Library Management System

Python project

GitHub Repository: https://github.com/ksenia180507/Library_management

Student name: Keseniia Goncharova

Student ID: GH1050782

Module: B100 Introduction to Computer Programming with Python

Table of contents

1.0 Introduction

This project was created for a library organization to provide a system which will help to manage books, members and borrowings. Libraries usually face difficulties when it comes to handling large amounts of data, such as registering members, recording borrowings and track book availability. Moreover, handling all these processes manually leads to a huge number of errors and data inconsistencies.

The purpose of the created software system is to provide a simple Library Management System that automates all the processes from registering new members to tracking borrowings. The program allows new users to register or already existing ones to log into the system, search for books, borrow and return books, at the same time librarians can manage all the borrowings and add new books to the stock.

Overall, the system includes user registration/login, searching book by title/author or genre, borrowing and returning books, managing book stock, managing borrowings records, save data to files and load data from files for a faster and easier processing. The software program is designed as a console-based app, which includes OOP (object oriented programming) concepts.

2.0 System design

The system is built around four core classes: Book, Member, Borrowings and Library, which represents the real life objects in a library system. Each class is in its own module, plus the fifth module is the main.py that coordinates these classes and provides interactive interface.

2.1 Classes structure

Book

It stores information about books including their book_id, title, genre, author, availability and quantity. The main methods of this class are change_stock(), which updates stock quantity, and matches(), which searches books according to the user's input, other two methods are default ones: **__init__(),** which is used to initialize variables at the beginning and **__str__**(), which returns a human-readable string representation of a class object.

Borrowings

This class stores borrowing_id, member_id, book_id and borrowing status. The main methods are: close_borrowing(), which changes the status of a borrowing to a “closed”, and match(), which returns True or False according to whether there is a match between the member and a book and whether its status is active. The other two methods are__**init**__() and __**str**__().

Member

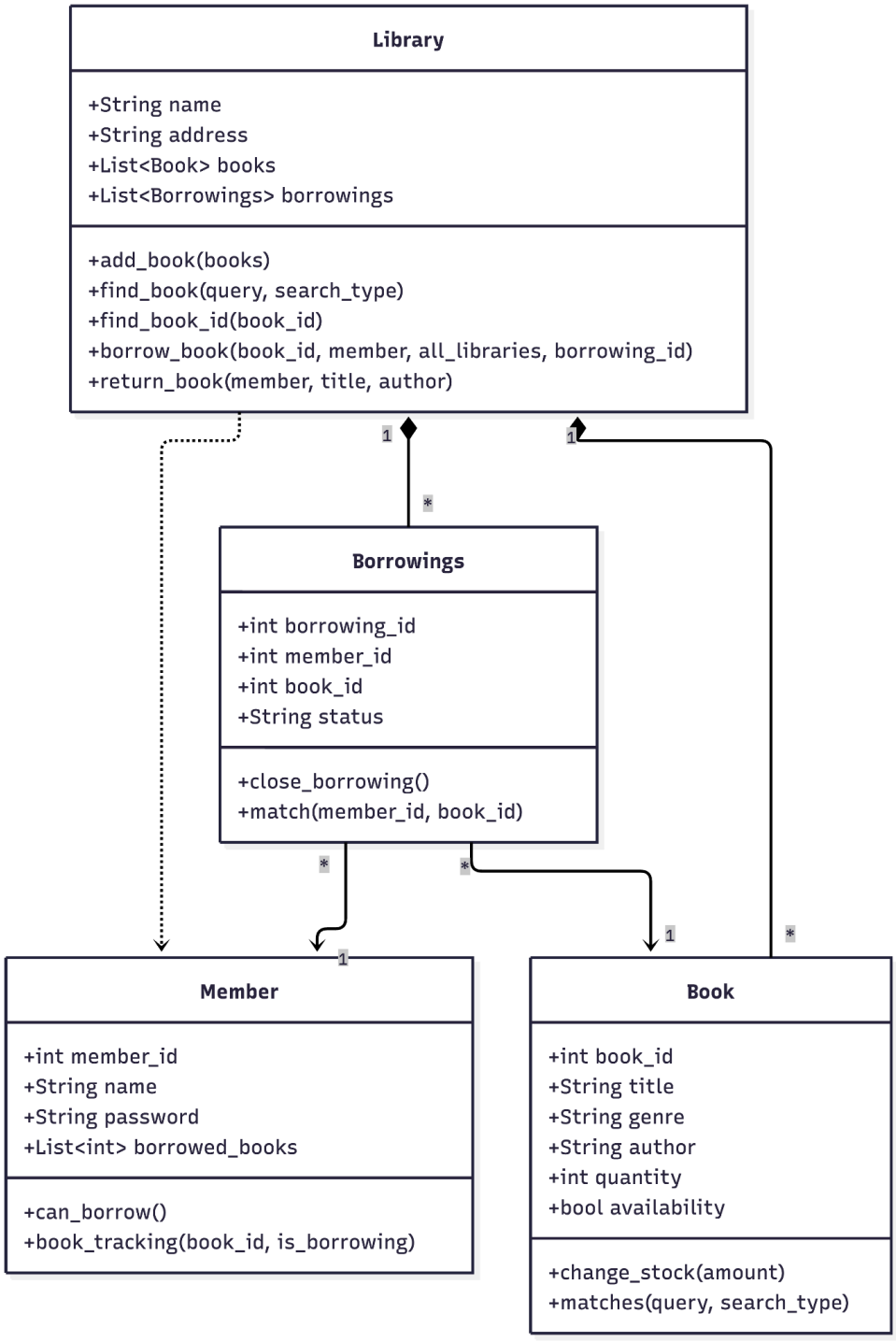
This class stores information about members including their member_id, name, password and borrowed books list. The main methods of the class are: can_borrow(), which returns True or False according to whether the member already has 10 borrowings or not, this will further track if a member can make borrowings. Another method is book_tracking, it updates the member’s list of borrowed books, it checks if book_id is not in the borrowing list to avoid data duplication and then allows borrowing the book, same with return, as it checks whether the book is in the list before its removal. The other two methods are__**init**__() and __**str**__() as well.

Library

This class is the main in the system, it stores information about the library (its name and address), books and borrowings lists. The main methods are the following. First one is add_book() it adds the book to a system, second one is find_book() it finds a book according to the user’s input and puts it in an empty list and then displays it, if the list remains empty it means that no books are found. The third function is find_book_id() it finds the book id, this function is used in another one for further processing. The fourth one is borrow_book() which finds a book (using the previous function), checks book availability, allocates the book to a member and changes the status and stock. The fifth function is return_book() it also finds a book (via find_book_id()), removes the book from a member, makes the book available by changing its status and stock. The other two methods are__**init**__() and __**str**__() as well.

2.2 Classes relationships

The classes are connected by ID, which means that each line of data maps to an object.



2.3 Data management

The program stores data using simple text file handling, which uses comma-separated files. There are three files: book.txt, members.txt and borrowings.txt. On startup, main.py loads each file into the corresponding in-memory objects (load_books_to_array(), load_members_data(), load_borrowings_array()) for easier and faster access to data while software is running. Whenever a book is borrowed or returned, a new member is registered, a librarian adds new books or closes active borrowings – the affected file is rewritten using save_books(), save_members() or save_borrowings() functions.

3.0 Implementation

3.1 Classes and Objects

Object oriented programming was used to create classes which represent real world entities within the library system. Each class is responsible for maintaining its own attributes and behavior. For example, class Book stores information related only to books, the Member stores information only about members etc.

For example:

```
  class Book: #stores data about a book
        def __init__(self, book_id, title, genre, author,
availability, quantity):
        self.book_id = book_id
        self.title = title
        self.genre = genre
        self.author = author
        self.quantity = quantity #stock of a book
        self.availability = availability
```

Encapsulation is used to ensure that data is managed consistently. For example, Book.matches() method allows to compare the title, author or genre within the Book class. This means that the Library.find_book() method does not need a direct access to these attributes, instead, it just asks whether there was a match with a user request/query or not.

Example:

Method in Book class

```
  def matches(self, query, search_type):
      """Matches the user input and existing data"""
       if search_type == "title" and query.lower().strip() in
self.title.lower():
          return True
      elif search_type == "author" and query.lower().strip() in
self.author.lower():
          return True
      elif search_type == "genre" and query.lower().strip() in
self.genre.lower():
          return True
      return False
```

Method in Library class:

```
  def find_book(self, query, search_type):
      """Function finds a book puts it in an empty list and then
displays it, if the list remains empty it means that no books
are found"""
      found_books = [] #empty list
      for b in self.books:
          if b.matches(query, search_type):
              found_books.append(b)

      return found_books
```

3.2 Functions and methods

Functions were implemented to separate tasks into reusable components. It is also made to make the code more organized and clearer as it reduces the repetition of the same blocks of code. Usage of functions also helps to easier debugging the program, as the code is divided into a small blocks

There are plenty of examples throughout the code.

Examples include:

- Data handling functions, such as: save_books(), save_members_file(), save_borrowings(), load_books_to_array(), load_members_data(), load_borrowings_array()
- System functions like: generate_member_id() or user_main_menu()
- Library management function such as: add_book(), find_book(), return_book()

3.3 Exception handling

Exception handling is used to prevent program crashes, by checking for invalid inputs from users using **try** and **except** blocks.

User inputs that require numeric value were enclosed within try-except blocks to allow the programme to run smoothly even if the input was incorrect.

Example of exception handling:

```
try:
    menu_choice = int(input("\nChoose an option: "))
except ValueError:
    print("Error: please enter a number")
    continue
```

Exception handling was implemented throughout the program, in parts such as: menu selection, quantity input, library selection etc. This ensures system reliability and user experience, as instead of program crashes the friendly message is displayed instead.

3.4 Modules

To improve code readability, the project was divided into separate files/modules. The whole project consists of:

- “book.py” contains book-related functionality
- “member.py” contains user related information
- “borrowings.py” handles borrowings records
- “library.py” manages all core library functions
- “main.py” contains main functionality such as user interaction, menus and file handling

By separating a project into smaller parts reduces code complexity and allows easier debugging, as a developer can also modify certain parts without affecting the whole system.

3.5 Control structures

Loops and conditional statements were used throughout the project code to control the program flow and implement the main logic.

The menu uses while loops and if/elif/else statements to continuously display options for the user and process its choices and requests. For example, the main menu repeatedly presents available options such as user login, registration, librarian mode and exiting the program. Same looping is used in the user menu, to display and process options such as, finding and borrowing a book, returning a book or exiting the user menu. Moreover, these menus also include conditional statements to set up, for example, a current library.

Example implementation:

```
    while True:
  # Exception handling is done in case of the input of incorrect
value
    try:
            lib_choice = int(input("\nEnter the number of the
library: "))

        if lib_choice == 1:
            current_library = library_1
            break

        elif lib_choice == 2:
            current_library = library_2
            break

        elif lib_choice == 3:
            current_library = library_3
```

```
            break

        else:
            # Checks if the number is in range
            print("Error: check the input number again!")

    except ValueError:
        #Checks if nothing or wrong information was input
        print("Error: please enter the number of a library")
```

Loops were also used in searching and borrowing operations. For example the function find_book() iterates through all books in a library to check whether the book matches the user's input.

```
for b in self.books:
    if b.matches(query, search_type):
        found_books.append(b)
```

So, looping and usage of conditional statements allows to make the system interactive and also ensures that all the decisions follow certain library logic/rules.

3.6 File handling

File handling using "txt" files was implemented to store all the data even after the program closes. In library system three files were used:

- "books.txt" stores book_id, title, genre, author, availability and quantity
- "members.txt" stores member_id, name and password
- "borrowings.txt" stores borrowing history which includes library name, borrowing_id, member_id, book_id and status of the borrowing.

The "with open()" structure was used to automatically close files after all operations are completed.

Example of data storage:

```
def save_members_file(all_members):
    #Saving members to members.txt
    with open("members.txt", "w",) as f:
        for m in all_members:
            f.write(f"{m.member_id},{m.name},{m.password}\n")
```

Example of data uploading to list:

```
def load_members_data(all_members):
    # exception handling, warns if file was not found
    try:
        with open("members.txt", "r") as f:
            for line in f:
                if line:
                    data = [item.strip() for item in
line.split(",")]
                    member = Member(int(data[0]), data[1],
data[2])

                    all_members.append(member)
        print("Members are loaded")

    except FileNotFoundError:
        print("Member file is not found")
```

3.7 Readability and Modularity

To ensure readability and modularity the project was divided into multiple files (see 3.4 Modules), which reduces code complexity. Readability was improved by using encapsulation and reducing duplication of code blocks (see 3.2 Functions and methods). Moreover, the design improves maintainability, as for example, Library class does not need to know certain attributes, which belong to Book (see 3.1 Classes and objects), and if any modifications are needed to be done, they will be made only in Book class.

Consistent naming is also used throughout the program. Variables and functions use snake_case naming, such as member_id or borrow_book(), while all classes use PascalCase, for example Member, Book or Library.

Moreover, comments and docstrings are used to better explain the code functionality and clarify all the operations.

Example:

```
def can_borrow(self):
    """User cannot borrow more than 4 books at a time"""
    return len(self.borrowed_books) < 4
```

4.0 Testing and demonstration

The program is tested manually by using a command-line interface and performing different scenarios. Testing covers both normal operations and error scenarios to check that all functions behave correctly. Different combinations of user registrations, logins, searchings, borrowings, returnings and librarian operations were tested. Error conditions such as invalid inputs, repeated passwords, and missing data files were tested to ensure that the system handles all the situations without crashing. The main purpose of testing is to ensure that all features run correctly and smoothly.

Which means that:

- User registration and login worked correctly
- Books could be searched successfully
- Borrowing and returning operations update data correctly
- File storage is consistent
- Exception handling is capable of preventing crashes
- All program rules are executed correctly (such as: borrowing limits or different access for users and librarians)

4.1 Test cases

The table below summarises the test cases performances and whether the output is the same as the expected result.

Test case	Input	Expected result	Output
Choosing the library program runs at	From 1-3	Library successfully chosen, all data below will be saved and processed regarding the chosen library	Success
User registration	Name: Fred Password: fred1234	New user is created and added to members.txt, the user menu then displayed	Success
User login	Name: John Password: john123 (existing member in library “database”)	Greeting is displayed and user gains access to a next menu	Success
Search for existing book in a current library	Choosing type of search: title, author or genre Input query according to the type of search (for example: Search_type: genre Query: Fiction)	Displaying the list of books in a current library	Success
Search for not existing book in a current library	Same procedure as above	Display the list of books in another libraries according to the user input or display a message, which says that there are no books in any library according to the user input.	Success
Borrow a book	Select the book from the list of available books	Quantity of a chosen book decreases by one and borrowing	Success

		record is created	
Return a book	Enter title and author of a borrowed book	Borrowing status changes to closed, quantity increases by one	Success
Login in librarian mode	Enter password	Checks the password if it is correct, program displays librarian menu, if not it warns that the password is wrong and brings user back to choice	Success
Adding new books to the system	Entering the number of different books arrived, enter the details of an each book	Adding books to the books.txt	Success
Manage borrowings as a librarian	Choosing the option to manage borrowings, if there are any, choosing an option to close the borrowing according to its ID	Successfully closing borrowing according to its ID, updating information in borrowing.txt If there are no borrowings, message displayed: “No borrowings”	Success
Invalid menu input	Enter “one” instead of a number	Error is displayed and option to enter the number of an option again	Success
Invalid library input	Enter the number of the library “one” instead of a number	Error is displayed and option to enter the number of a library again	Success
Password duplication in registration	Enter, while registering, password: “john123” (existing one)	This password is occupied, option to register again	Success

Maximum borrowing per person	Try creating more than 4 “active” borrowings per one person	Displays the error, as 1 person cannot have more than 4 “active” borrowings. The system prevents new borrowing.	Success
Returning book a person has no borrowings	Press return book using member account, which has no active borrowings	System displays: there are no active borrowings	Success
Missing data files on first program run	Run program without one of the files like book.txt	Warning message is shown, but program continues execution	Success

5.0 Reflection

5.1 Challenges encountered

The main challenge which was encountered while elaborating the code for the project is maintaining consistency between different parts of the system when operations such as borrowing and returning occur, as several components have to be updated like stock, member’s borrowing list and the borrowing records. Another challenge was to correctly design all the classes and their functionality, good example of search functionality inside Library class, as the whole method could have been implemented only to Library class, but in this case title, authors and genres also should be there. However, implementing part of the searching logic to Book.matches() is a better option, as in this case there is no need to give Library class additional data, as a result it improves maintainability.

5.2 Lesson learned

This project allowed me to improve my object-oriented programming skills and gave me a greater understanding of coding concepts. Moreover, dividing the project into smaller parts made implementation much easier to understand, as separating the code allowed to reduce the complexity and develop all the components individually.

Developing a library system required careful planning of all the operations and main logic, so this project improved my logical thinking in order to make all the operations work correctly and consistently.

Overall, I learned that planning the relationships between classes and functions before implementation helps to reduce error and makes the development process easier.

5.3 Future improvements

Although the program successfully achieves its main objectives, it is still not ideal and cannot be implemented in real library, as more improvement should be made

Improvements:

- Replacing simple file handling with csv files or database using SQL
- Adding “datetime” for better borrowings management, for example can be used to set deadlines when the borrowing has to be returned, after this the system can be improved by adding fines for overdue
- Improving searching tab by developing better filtering options
- “Tkinter” library can be used to improve interface by creating graphical user interface, this will improve user experience
- A reservation system also can be added, to allow users create a queue for a book which is currently unavailable.

To conclude, the project itself demonstrates all necessary requirements like object_oriented programming, file handling, exception handling, looping and conditional statements, and inputs and outputs operations.